

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS, 2027-1
INTRODUCCIÓN A CIENCIAS DE LA COMPUTACIÓN



PRÁCTICA 05:
Autómatas Celulares

PROFESOR:
Manuel Alcántara Juárez

AYUDANTE DE TEORÍA:
Ángel Renato Zamudio Malagón

AYUDANTE DE LABORATORIO:
Rodrigo Alejandro Sánchez Morales

Práctica 05: Autómatas Celulares

Objetivo:

Construir un programa que simule un autómatas celular. El programa debe poder mostrar el inicio de la configuración (dependiendo de los estados iniciales que se piden) y mostrar cada generación (más adelante se mostrarán los detalles para calcularlas).

Instrucciones:

¿Qué es un Autómata Celular?

Un autómatas celular es un modelo matemático y computacional que simula cómo evolucionan los sistemas complejos a partir de reglas muy simples. Imagina una cuadrícula (como un tablero de ajedrez o una hoja cuadriculada) donde cada cuadrado es una celda.

Cada celda puede estar en un estado específico. En el modelo más famoso, el "*Juego de la vida*" (creado por el matemático John Conway en 1970), las celdas solo tienen dos estados posibles: **Vivas** o **Muertas**.

El tiempo en este mundo no fluye de forma continua, sino en pasos discretos llamados **Generaciones**. Para calcular qué le pasa a una celda en la siguiente generación, esta debe mirar a sus 8 celdas vecinas (horizontales, verticales y diagonales) y aplicar 4 reglas básicas:

- Soledad (Subpoblación):** Toda celda viva con menos de 2 vecinos vivos, muere.
- Supervivencia (Estabilidad):** Toda celda viva con 2 o 3 vecinos vivos, sigue viva en la siguiente generación.
- Sobrepoblación (Masificación):** Toda celda viva con más de 3 vecinos vivos, muere.
- Reproducción (Nacimiento):** Toda celda muerta con exactamente 3 vecinos vivos, "nace" (se vuelve viva).

No hay jugadores. Una vez que configuras el patrón inicial en el "*Juego de la vida*" de celdas vivas y muertas, el sistema evoluciona por sí solo.

Para comprender el comportamiento del "*Juego de la vida*", debemos aislar la celda central de una cuadrícula de 3x3 y observar cómo interactúa con sus 8 vecinos directos.

A continuación, se darán ejemplos de cada una de las reglas para que queden más claros los casos con la siguiente simbología.

Simbología:

- Celda Viva: ■
- Celda Muerta: □
- Celda Objetivo: ☒ o ☒ (La celda central que estamos evaluando, marcada con una palomita).

Regla 1: Soledad (Subpoblación)

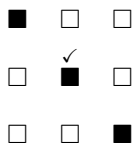
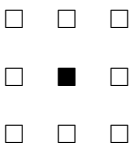
Toda celda viva que tenga **menos de 2 vecinos vivos** (es decir, 0 o 1) morirá en la siguiente generación por falta de apoyo o aislamiento.

Estado Actual (Generación 1)	Resultado (Generación 2)
□ □ □ □ ☒ □ □ □ ■	□ □ □ □ □ □ □ □ □

Análisis: La celda central viva (*objetivo*) solo tiene **1 vecino** vivo (en la esquina inferior derecha). Al calcular el siguiente turno, la celda central muere por soledad. (*Nota: El vecino de la esquina también muere porque se queda con 0 vecinos*).

Regla 2: Supervivencia (Estabilidad)

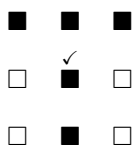
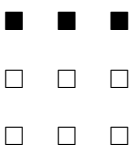
Toda celda viva que tenga exactamente **2 o 3 vecinos vivos** sobrevivirá y pasará a la siguiente generación sin cambios. Es el equilibrio perfecto.

Estado Actual (Generación 1)	Resultado (Generación 2)
	

Análisis: La celda central viva tiene exactamente **2 vecinos** vivos (las esquinas superior izquierda e inferior derecha). Al avanzar el tiempo, la celda central se mantiene viva. *(Las celdas de las esquinas mueren por la Regla 1 al tener solo 1 vecino).*

Regla 3: Sobrepoblación (Masificación)

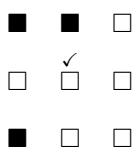
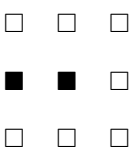
Toda celda viva que tenga **más de 3 vecinos vivos** (4 a 8 vecinos) morirá en la siguiente generación por asfixia o falta de recursos.

Estado Actual (Generación 1)	Resultado (Generación 2)
	

Análisis: La celda central viva está rodeada por **4 vecinos** vivos (los tres de arriba y el de abajo). Al ser demasiados, el sistema colapsa en ese punto y la celda central muere por sobrepoblación. *(Las tres celdas de arriba sobreviven porque se apoyan mutuamente formando una línea).*

Regla 4: Reproducción (Nacimiento)

Toda celda **muerta** que esté rodeada por **exactamente 3 vecinos vivos** nacerá y se convertirá en una celda viva en la siguiente generación.

Estado Actual (Generación 1)	Resultado (Generación 2)
	

Análisis: La celda central está **muerta**. Sin embargo, al tener exactamente **3 vecinos** vivos a su alrededor (dos arriba y uno en la esquina inferior izquierda), se cumplen las condiciones para la reproducción. En la Generación 2, la celda central **nace** (■). *(Simultáneamente, los vecinos originales mueren por falta de apoyo, pero nace una nueva celda a la izquierda de nuestra celda central por tener también 3 vecinos).*

Implementación en Java

1. Arreglos:

El tablero: Usaremos una matriz de tipo `boolean[][]`. Un valor `true` significará que la celda está viva, y `false` que está muerta.

2. Interacción y Ciclos (`for`, `while`, `do-while`, `switch`):

El tiempo: Usaremos un ciclo para calcular y mostrar un número determinado de generaciones (por ejemplo, 100 pasos).

3. Estructuras de decisión (`if-else`):

Las reglas: Crearemos una función que cuente los vecinos de cada coordenada. Luego, usaremos condiciones `if` para decidir si la celda vive o muere.

4. El problema de la simultaneidad:

Este es el concepto más importante: Si actualizamos el tablero original celda por celda, las celdas finales se calcularán usando los datos nuevos de sus vecinos, no los antiguos. Para que las reglas se apliquen a todas las celdas al mismo tiempo, crearemos un segundo arreglo bidimensional (un tablero temporal). Escribiremos los nuevos estados en el tablero temporal y, al final de la generación, reemplazaremos el tablero viejo por el nuevo.

5. Comentarios:

Al ir desarrollando tu código, agrega **comentarios de documentación** en las partes importantes y generalizadas del funcionamiento de tu código.

6. Funciones:

Se pide que crees **funciones** para:

- Imprimir el tablero actual y su estado.
- Calcular la siguiente generación.
- Contar los vecinos vivos de la celda actual.

7. Constantes:

Creas **constantes** con los valores que no van a cambiar a lo largo del programa.

Entregables de la práctica

Deberás subir a tu repositorio:

- i. Para la elaboración de esta práctica, se puede entregar en equipos de dos personas.
- ii. **Código fuente:** El texto de tu programa `AutomatasCelulares.java`. programa, con los resultados visibles.
- iii. El archivo `README.md` modificado (tendrás que poner tu nombre, número de cuenta y correo institucional).
- iv. Deberás subir todos sus archivos mencionados anteriormente a *Github Classroom*. Debe de estar organizado de la siguiente manera, (suponiendo que quien está entregando es el ayudante de laboratorio).

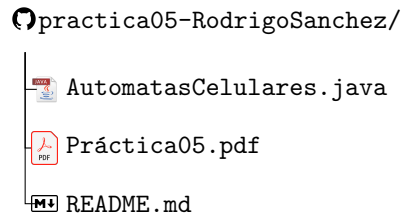


Figura 1: *Entregables.*

Nota.

Para cualquier duda o comentario que pudiera surgirles al hacer este trabajo, recuerden que cuentan con la plataforma de *Classroom*, donde pueden hacer todas las preguntas que necesiten.



Figura 2: *Nota.*